



Chapter 12 Network Programming



Outline (Chapter 12)

- **Basic principles of network communication and IP addresses**
- **Socket programming library**
- **Use of URLs**
- **Observer pattern**



Network communication

The main purpose of network programming is to communicate directly or indirectly with other computers through network protocols. There are two main problems in network programming:

1. How to accurately locate one or more hosts on the network. ➡ **IP address, port number, etc**
2. How to transmit data reliably and efficiently. ➡ **Network Transport Protocol**

This section begins with a review of **IP addresses** and **network transport protocols**.





IP address

- ❑ In the Internet, an IP address is used to uniquely identify a network interface. A computer connected to the Internet will definitely have one IP address, but it can also have multiple IP addresses. So, if a machine has a network card, then it will have two addresses, namely the local address and the IP address.
- ❑ There are two types of IP addresses: IPv4 and IPv6.
 - IPv4 uses 32-bit addresses, and IPv4 addresses are currently exhausted, such as 101.202.99.12
 - IPv6 has 128-bit addresses, and the addresses are not used at all, such as
2001:0DA8:100A:0000:0000:1020:F2F3:1428
- ❑ There is also a special IP address, called the local address, which is 127.0.0.1.



Domain

IP addresses are not easy to remember by numbers, and TCP/IP has designed a mechanism for characters to represent addresses in characters, called the Domain Name System (DNS).

A complete domain name generally is:

hostname.localname.groupname.top-level-domain

For example, a website domain name:

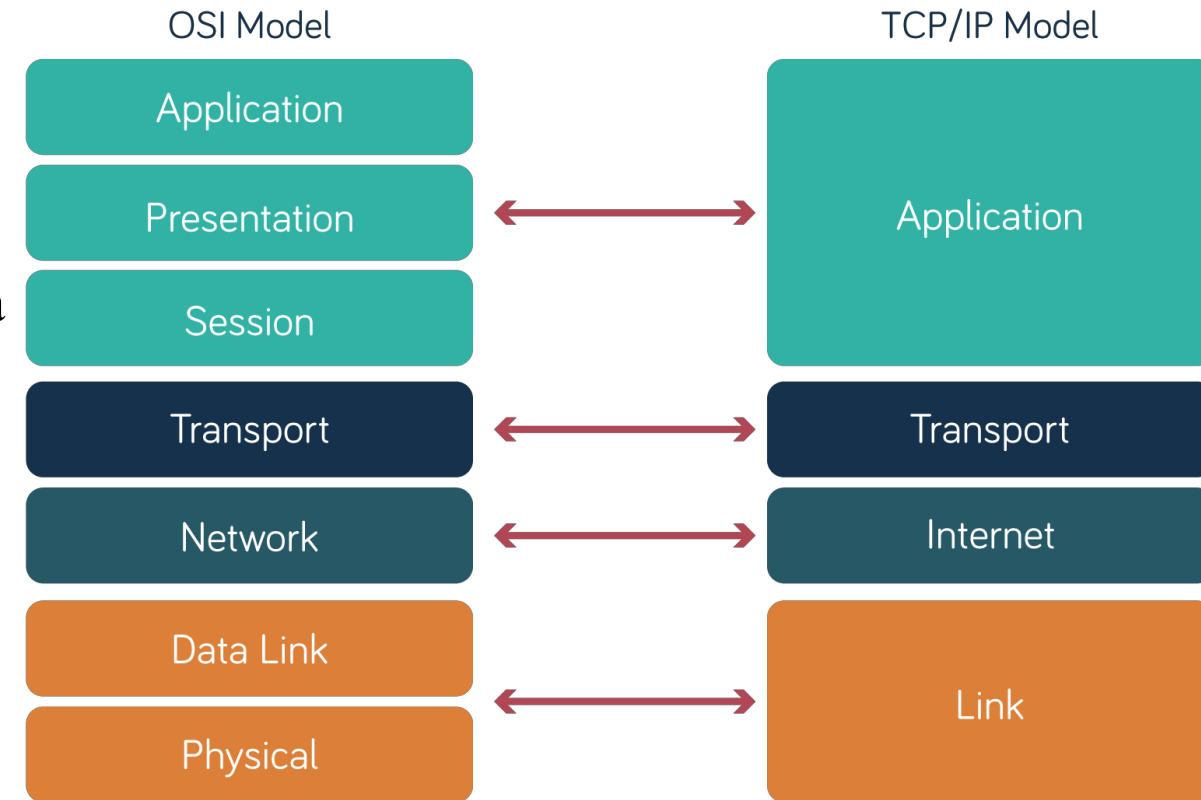
IP: 112.80.248.74

Domain: www.baidu.com



Network protocol

- In 1977, the International Organization for Standardization (ISO) established a special agency and proposed a standard framework for computers to be interconnected into networks around the world, namely the famous basic reference model for open system interconnection, referred to as the OSI model, which is a theoretical model.
- The TCP/IP protocol is used to communicate between computers on the Internet, which consists of four layers: application layer, transport layer, Internet layer, and network interface layer. Each layer in turn includes a number of agreements. Writing network communication programs in Java is generally at the application layer.





Network protocol

Support for two common network protocols is provided in java.net package:

- TCP: TCP (Transmission Control Protocol) is a **connection-oriented, reliable, byte-stream-based** transport layer communication protocol that is above the IP layer and below the application layer. TCP guarantees reliable communication between two applications. It is commonly used for the Internet protocol and is known as TCP/IP.
- UDP: UDP (User Datagram Protocol) is located at the transport layer of the OSI model. A connectionless protocol. Datagrams are provided to send data between applications. Because UDP **lacks reliability** and is a connectionless protocol, applications often have to tolerate some lost, erroneous, or duplicate packets.



Outline (Chapter 12)

- **Basic principles of network communication and IP addresses**
- **Socket programming library**
- **Use of URLs**
- **Observer pattern**



Socket

- ❑ Socket is an abstract concept that uses the TCP protocol for communication. Sockets use TCP to provide a communication mechanism between two computers, allowing programmers to **treat network connections as a stream**. The client program creates a socket and tries to connect to the socket of the server.

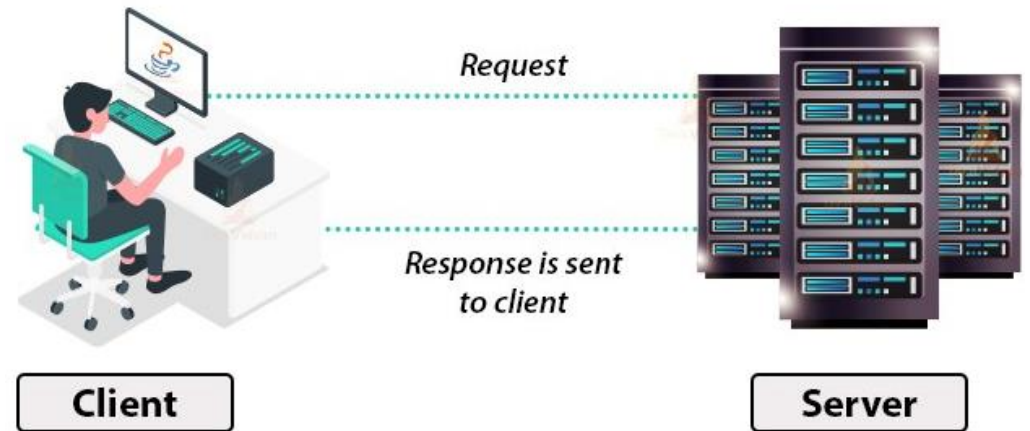
- ❑ Sockets are uniquely determined by a port number and an IP address.



Application



Computer

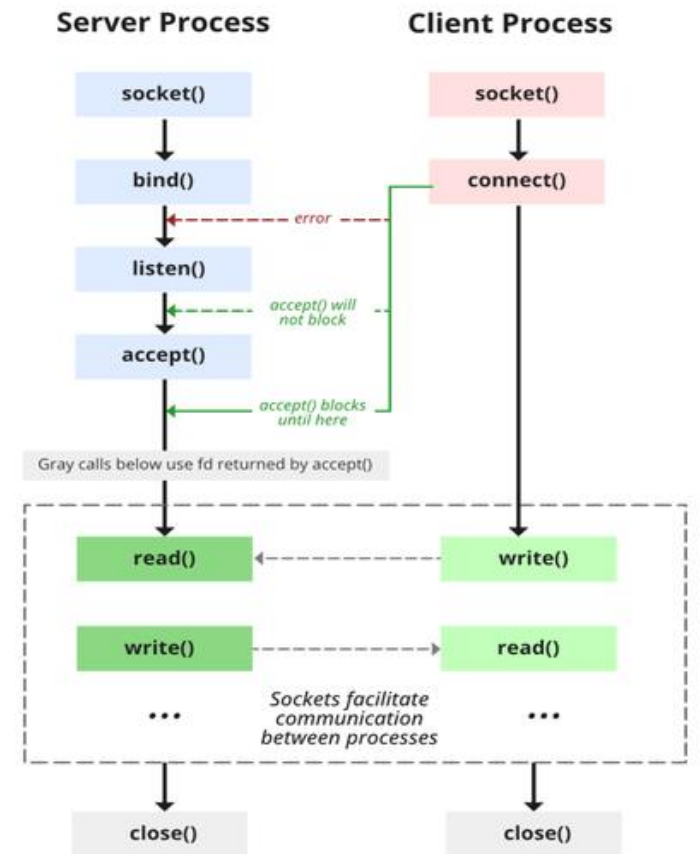




Socket

The following steps appear when a TCP connection is established between two computers using a Socket:

- The server instantiates a ServerSocket object
- The server calls the accept() method of the ServerSocket class and waits for the client to connect.
- The client attempts to instantiate a socket object specifying the server name and port number to request a connection.
- Once the connection is established, the server gets a new Socket object and communicates with the client using the I/O stream.





Server-side

An example of a server-side creation of a ServerSocket:

```
public class Server {  
    public static void main(String[] args) throws IOException {  
        ServerSocket ss = new ServerSocket(6666);  
        System.out.println("server is running...");  
        for (;;) {  
            Socket sock = ss.accept();  
            System.out.println("connected from " + sock.getRemoteSocketAddress());  
            Thread t = new Handler(sock);  
            t.start();  
        }  
    }  
}
```

The server listens to all IP addresses of the server, port 6666. After establishing a connection via `accept()`, the obtained `Socket` is sent into the `Handler` for subsequent operations.



Client

The client creates a socket, connects with the server, obtains the input and output stream, feeds it into the handle, and transmits it:

```
public class Client {  
    public static void main(String[] args) throws IOException {  
        Socket sock = new Socket("localhost", 6666);  
        try (InputStream input = sock.getInputStream()) {  
            try (OutputStream output = sock.getOutputStream()) {  
                handle(input, output);  
            }  
        }  
        sock.close();  
        System.out.println("disconnected.");  
    }  
}
```

Note: The server in the example is "localhost", which transmits with native port 6666.

Note: After the server establishes a connection with the client, it also uses Socket instead of ServerSocket for transmission.



Socket flow

When a Socket connection is successfully created, both the server and the client use the Socket instance for network communication.

Because TCP is a stream-based protocol, the Java standard library uses `InputStream` and `OutputStream` to encapsulate the data flow of a socket, similar to a normal IO stream:

```
// Read network data
InputStream in = sock.getInputStream();
// Write network data
OutputStream out = sock.getOutputStream();
```

The information sent by the server to the server output stream becomes input to the client. After that, all output from the client program is included in the server's input stream.



Outline (Chapter 12)

- **Basic principles of network communication and IP addresses**
- **Socket programming library**
- **Use of URLs**
- **Observer pattern**



URL

What is a URL?

URL: Uniform Resource Locator. Used to **read resources from the host** (only read, not write to the host).

A URL address usually consists of 4 parts:

- Protocol name: such as http, ftp, file, etc
- Host name: such as baidu, 220.181.112.143, etc
- Pathway files: e.g. /java/index.jsp
- Port number: such as 8080, 8081, etc



URL

URL example:

`https://zh.wikipedia.org:443/w/index.php?title=Uniform_Resource_Locator`

- `https`, is the protocol;
- `zh.wikipedia.org` is the server;
- `443`, is the network port number on the server;
- `/w/index.php`, is the path;
- `?title=Uniform_Resource_Locator`, is an inquiry

To communicate using URLs, you need to use URL classes to create objects and complete network communication by referencing URL classes. Create a URL object to reference the construction method of the `java.net.URL` class provided in `java.net` package.



Create an object for the URL class

URL classes provide 4 construction methods for creating URL objects:

➤ **1. public URL(String str)**

It is to create a URL object using a string of URLs. As:

```
URL myurl = new URL( "http://www.edu.cn" );
```

➤ **2. public URL(String protocol, String host, String file)**

In this construction method, the protocol name "protocol", the hostname "host", and the file name "file" are specified, and the port uses the default value.

```
URL myurl = new URL("http", "www.edu.cn", "index.html");
```



Create an object for the URL class

➤ 3. **public URL(String protocol, String host, String port, String file)**

This construction method has an additional port number "port" compared to the second construction method.

```
URL myurl = new URL("http", "www.edu.cn", "80", "index.html");
```

➤ 4. **public URL(URL content, String str)**

This construct gives a path offset relative to content. As:

```
URL mynewurl = new URL(myurl, "setup/local.html");
```

The URL object myurl is the previous URL object, so the URL address represented by mynewurl is:

<http://www.edu.cn:80/setup/local.html>



Use JAVA to get web pages via URL

```
import java.io.*;
import java.net.URL;

public class Main {
    public static void main(String[] args) throws Exception {
        URL url=new URL("https://www.baidu.com/");
        BufferedReader reader=new BufferedReader(new InputStreamReader(url.openStream()));
        BufferedWriter writer=new BufferedWriter(new FileWriter("info.html"));
        String line;
        while((line = reader.readLine()) != null){
            System.out.println(line);
            writer.write(line);
            writer.newLine();
        }
        reader.close();
        writer.close();
    }
}
```



The difference between URL and Socket

They are differentiated by:

- The socket communication method is to run the communication program on the server side, constantly listen to the client's connection request, actively wait for the client's request service, and immediately connect and communicate when the client makes a request. When the URL communicates, it passively waits for the client request.
- Socket communication is that the server can communicate with multiple clients at the same time, while URL communication is that the server can only communicate with one customer.



Outline (Chapter 12)

- **Basic principles of network communication and IP addresses**
- **Socket programming library**
- **Use of URLs**
- **Observer pattern**



Observer pattern

Pattern motivation

- **Software system:** A change in the state or behavior of an object will lead to a change in the state or behavior of other objects, and there will be a linkage between them
- **Observer Mode:**
 - It defines a one-to-many dependency between objects, so that changes in one object can affect other objects
 - The object that changes is called the observation target, and the object that is notified is called the observer
 - An observation target can correspond to multiple observers



Observer pattern

Definition

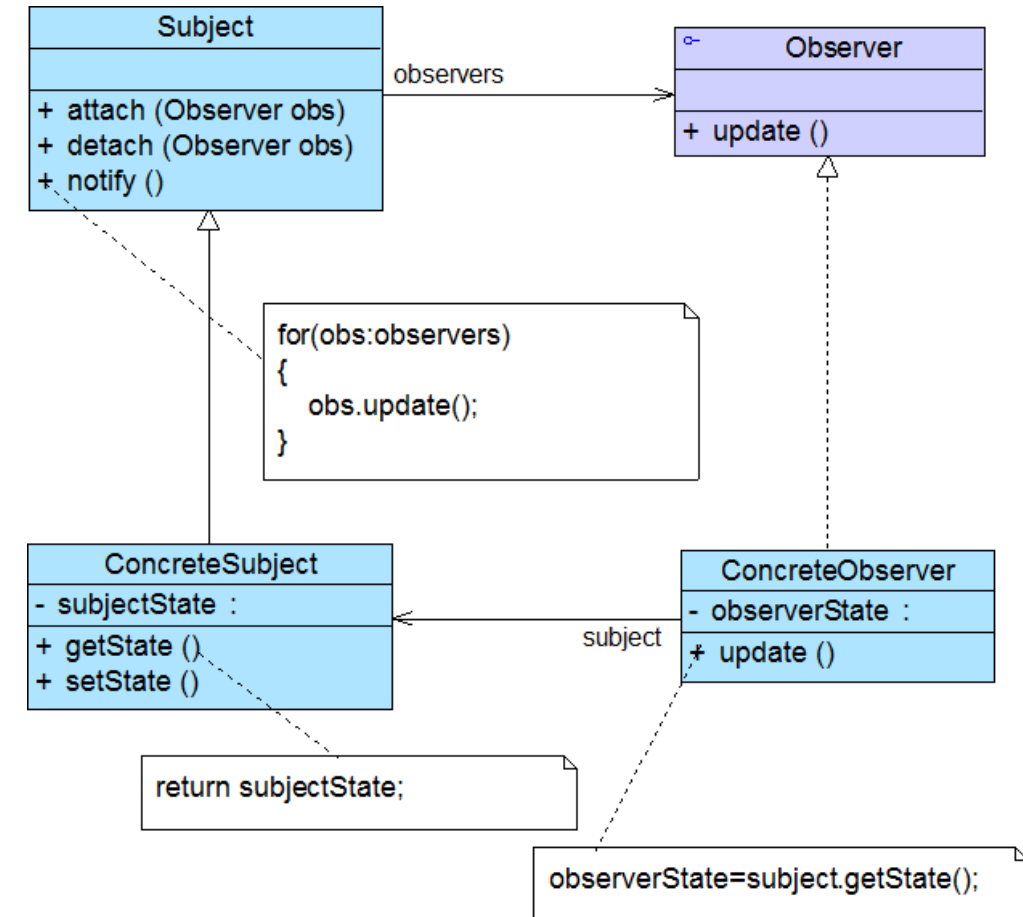
- Observer Pattern: Defines a one-to-many dependency between objects, so that whenever an object's state changes, its related dependent objects are notified and automatically updated.
- The observer mode is also called the Publish/Subscribe pattern, the Model/View pattern, the Source/Listener pattern, or the Dependents pattern
- The observer pattern is an object behavioral pattern



Observer pattern

Observer mode includes the following characters:

- Abstract Target (subject): Keep all references to observer objects in a set, and each abstract object can have any number of observers. Abstract targets provide an interface to add and remove observer roles. It is generally implemented with an abstract class and interface.
- Abstract observer: Define an interface for all concrete observers and update themselves when notified of the target.
- Concrete Subject: To notify all registered observers when the status of a specific topic changes within it.
- Concrete Observer: This role implements the updated interface required by the abstract observer role to harmonize its own state with the state of the subject. Usually implemented with a subclass. If desired, the specific observer role can save a reference to the specific subject role.





Observer pattern

Analysis: Abstract target class sample code:

```
import java.util.*;
public abstract class Subject {
    //Define a collection of observers to store all observer objects
    protected ArrayList<Observer> = new ArrayList();

    //Registration method to add an observer to the observer collection
    public void attach(Observer observer) {
        observers.add(observer);
    }

    //Logout method to remove an observer from the observer collection
    public void detach(Observer observer) {
        observers.remove(observer);
    }

    //Declare an abstract notification method
    public abstract void notify();
}
```



Observer pattern

Pattern analysis

➤ Target Class Sample Code:

```
public class ConcreteSubject extends Subject {  
    // Implement notification methods  
    public void notify() {  
        // Traverse the collection of observers and call the  
        response method of each observer  
        for(Object obs:observers) {  
            ((Observer)obs).update();  
        }  
    }  
}
```



Observer pattern

Pattern examples

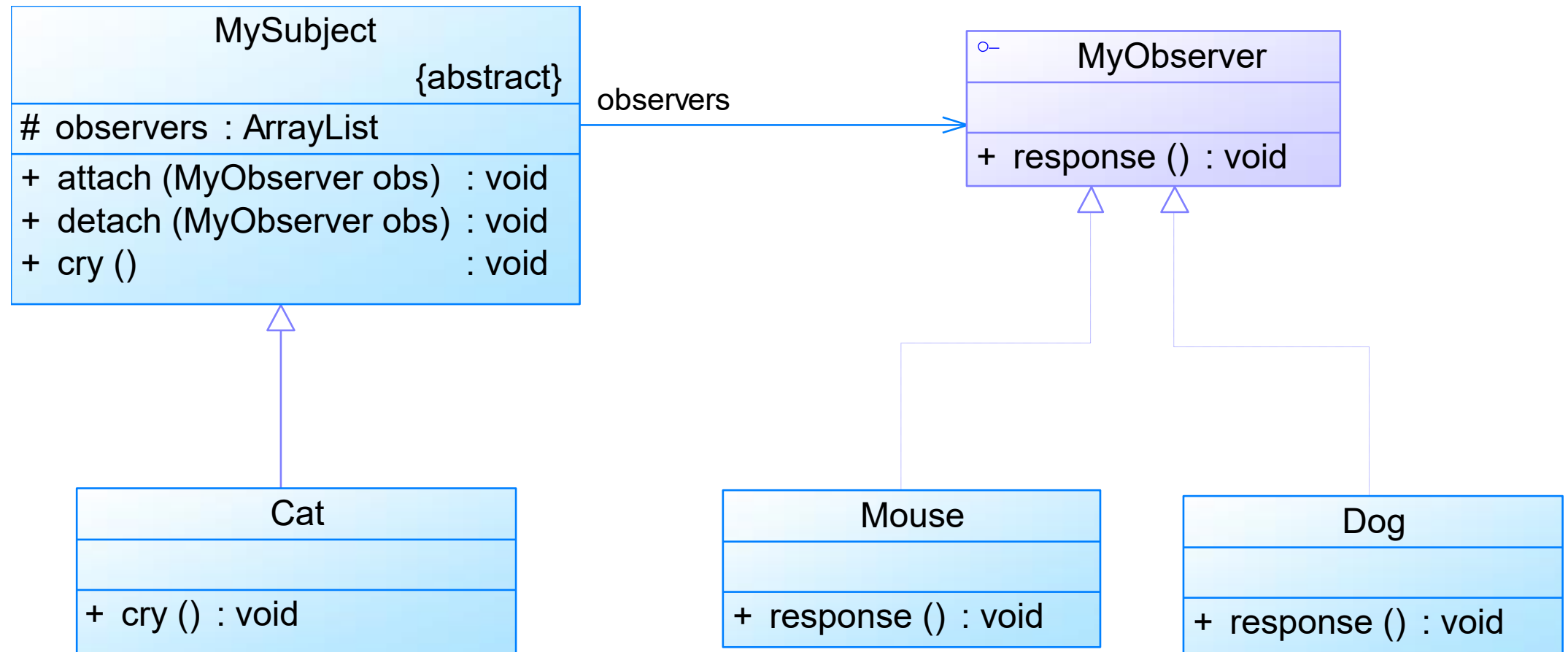
- Cats, dogs and mice: illustrated examples
 - Assuming that the **cat is the observation target, and the mouse and dog are the observers**, the cat calls the mouse to run and the dog follows, describing the process using observer mode.



Observer pattern

Pattern examples

- Cats, dogs and mice: Reference class diagrams





Observer pattern

Abstract Target

```
public abstract class MySubject
{
    protected ArrayList observers = new ArrayList();

    //How to register
    public void attach(MyObserver observer)
    {
        observers.add(observer);
    }

    //How to log out
    public void detach(MyObserver observer)
    {
        observers.remove(observer);
    }

    public abstract void cry(); //Abstract notification method
}
```



Observer pattern

Specific Target—Cat

```
public class Cat extends MySubject
{
    public void cry()
    {
        System.out.println("Mew!");
        System.out.println("-----");

        for(Object obs:observers)
        {
            ((MyObserver)obs).response();
        }
    }
}
```



Observer pattern

Observer interface

```
public interface MyObserver
{
    void response(); //Abstract response methods
}
```



Observer pattern

Specific observers - dogs and mice

```
public class Dog implements MyObserver
{
    public void response()
    {
        System.out.println("The dog barked!");
    }
}
```

```
public class Mouse implements MyObserver
{
    public void response()
    {
        System.out.println("The rat tries to escape!");
    }
}
```



Observer pattern

Test

```
public class Client
{
    public static void main(String[] args)
    {
        MySubject subject=new Cat();

        MyObserver obs1,obs2,obs3;
        obs1=new Mouse();
        obs2=new Mouse();
        obs3=new Dog();

        subject.attach(obs1);
        subject.attach(obs2);
        subject.attach(obs3);

        subject.cry();
    }
}
```



Observer pattern

Observer pattern advantages:

- It can be separated from the presentation layer and the data logic layer
- Establish an abstract coupling between the subject and the observer
- Supports broadcast communication, simplifying the difficulty of designing one-to-many systems
- In line with the open/close principle, adding a new specific observer does not need to modify the original system code, and it is also convenient to add a new observation target when there is no correlation between the specific observer and the observation target

Observer pattern disadvantages:

- It will take a lot of time to notify all observers
- If there are loop dependencies, it can cause the system to crash
- There is no corresponding mechanism for the observer to know how the subject has changed, but only to know that the object of observation has changed



Observer pattern

Observer mode can be used in the following cases:

- When you need to separate dependent aspects of a model into independent, reusable objects
- A change in an object will cause one or more other objects to change, and it is not known exactly how many objects will change or who they are
- A trigger chain needs to be created in the system



Questions

Problem: You're building an order system where when an order is placed, multiple actions must happen:

Update inventory

Send confirmation email

Process payment

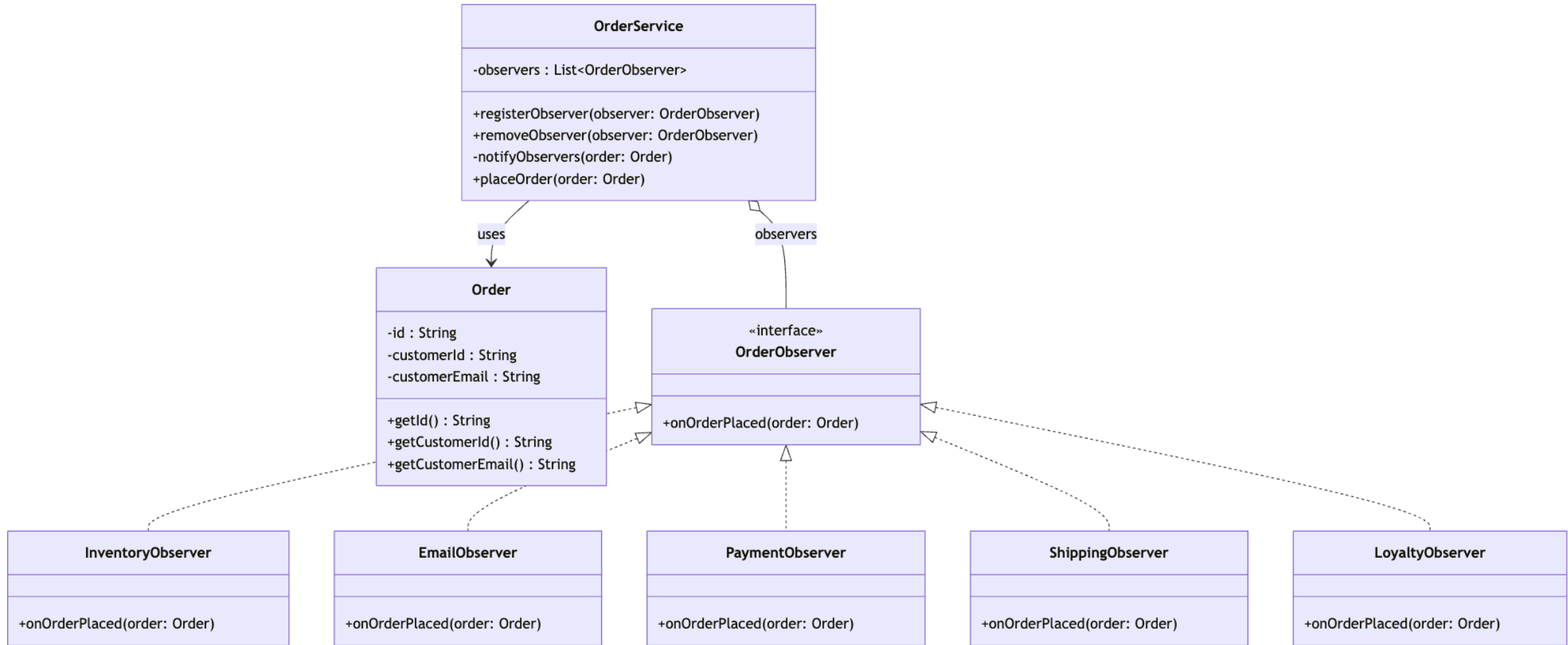
Notify shipping department

Update customer loyalty points

Design class diagram and implement.



Questions





Questions

```
class Order {  
    private final String id;  
    private final String customerId;  
    private final String customerEmail;  
  
    public Order(String id, String customerId, String customerEmail) {  
        this.id = id;  
        this.customerId = customerId;  
        this.customerEmail = customerEmail;  
    }  
  
    public String getId() {  
        return id;  
    }  
  
    public String getCustomerId() {  
        return customerId;  
    }  
  
    public String getCustomerEmail() {  
        return customerEmail;  
    }  
}
```



Questions

```
interface OrderObserver {
    void onOrderPlaced(Order order);
}

class InventoryObserver implements OrderObserver {
    @Override
    public void onOrderPlaced(Order order) {
        System.out.println("Inventory updated for order " + order.getId());
    }
}

class EmailObserver implements OrderObserver {
    @Override
    public void onOrderPlaced(Order order) {
        System.out.println("Confirmation email sent to " + order.getCustomerEmail());
    }
}
```



Questions

```
class PaymentObserver implements OrderObserver {
    @Override
    public void onOrderPlaced(Order order) {
        System.out.println("Payment processed for order " + order.getId());
    }
}

class ShippingObserver implements OrderObserver {
    @Override
    public void onOrderPlaced(Order order) {
        System.out.println("Shipping department notified for order " + order.getId());
    }
}

class LoyaltyObserver implements OrderObserver {
    @Override
    public void onOrderPlaced(Order order) {
        System.out.println("Loyalty points updated for customer " +
order.getId());
    }
}
```



Questions

```
class OrderService {  
    private List<OrderObserver> observers = new ArrayList<>();  
  
    public void registerObserver(OrderObserver observer) {  
        observers.add(observer);  
    }  
  
    public void removeObserver(OrderObserver observer) {  
        observers.remove(observer);  
    }  
  
    private void notifyObservers(Order order) {  
        for (OrderObserver observer : observers) {  
            observer.onOrderPlaced(order);  
        }  
    }  
  
    public void placeOrder(Order order) {  
        // core responsibility: save order  
        System.out.println("Order saved: " + order.getId());  
  
        // after core work, notify all observers  
        notifyObservers(order);  
    }  
}
```



Questions

```
public class Main {  
    public static void main(String[] args) {  
        OrderService orderService = new OrderService();  
  
        // register observers  
        orderService.registerObserver(new InventoryObserver());  
        orderService.registerObserver(new EmailObserver());  
        orderService.registerObserver(new PaymentObserver());  
        orderService.registerObserver(new ShippingObserver());  
        orderService.registerObserver(new LoyaltyObserver());  
  
        // create an order  
        Order order = new Order("O-1001", "C-001", "customer@example.com");  
  
        // place order: this triggers all observers  
        orderService.placeOrder(order);  
    }  
}
```



Outline (Chapter 12)

- **Basic principles of network communication and IP addresses**
- **Socket programming library**
- **Use of URLs**
- **Observer pattern**